

CSS

Cascading Style Sheets

Grundlagen, zum Bau der ersten eigenen Website mit Hypertext Markup Language und Cascading Style Sheets.

Ihr Dozent: Robert Reich



Agenda

Was wird mir in dieser Woche vermittelt?

- Grundlagen
- Syntax
- Formatierung
- Layout
- Erstellung einer eigenen Website
- Responsive Design

Nützliche Resourcen

- CSS
- Mozilla Dev https://developer.mozilla.org/
- Stack Overflow https://stackoverflow.com/
- W3schools.com https://www.w3schools.com/
- CSS-Tricks https://css-tricks.com/
- HTML-Seminar https://www.html-seminar.de/

- Freie Bilder: Unsplash.com https://unsplash.com/
- Skizziertools: https://excalidraw.com/ & https://figma.com
- https://codesandbox.io/

Geschichte von CSS

- Das Konzept der Cascading Style Sheets (CSS) wurde 1994 von Håkon Wium Lie vorgeschlagen. Bert Bos arbeitete zu dieser Zeit an der Entwicklung eines Darstellungsprogrammes namens Argo, der seine eigene Stilvorlagensprache benutzte. Die beiden entschieden sich zusammenzuarbeiten und CSS zu entwickeln.

Geschichte von CSS

- Im Dezember 1996 wurde die CSS Level 1 publiziert.
- Aktuell ist CSS3

Hintergedanke von CSS

- Trennung von Inhalten und Design
- Zeitersparnis bei Änderungen

CSS – Themenübersicht

1. Einstieg: Erste Schritte beim „stylen“ von Selektoren
2. Welche Selektoren gibt es?
3. Box Model
4. Kombinatoren (Einschränken der Wirkungsreichweite)
5. Kaskadierende Styles („cascading“)

CSS – Thema: Einstieg

Ein Selektor, gefolgt von einer geschwungenen Klammer mit einer Anzahl an Styles, die sich in Property : Value aufteilen.

```
body {
    font-family: Arial, sans-serif;
    font-size: 16px;
    color: black;
}
```

CSS – Thema: Selektoren

Welche Selektoren gibt es?

- Typselektoren (HTML tags, z.B. body, p oder h1 bis h6)
- Universalselektoren (das Sternchen * was alles auswählt)
- Klassenselektoren (Startet in CSS mit einem Punkt vorweg)
- ID-Selektoren (Startet in CSS mit einem Hashtag # vorweg)

CSS – Thema: Kombinatoren

Wir begrenzen die Wirkung von Styles auf einen Container, in den meisten Fällen mit einem Klassenselektor.

Nachfahrenselektor (einfach ein Leerzeichen)

```
.navigation   a  { color: blue; }
```

Wir stylen einen Link <a>, der aber nur angesprochen wird, wenn er in einem Container mit der Klasse .navigation liegt.

CSS – Thema: Kombinatoren

CSS

```css
a { color: green; }
.navigation a  { color: blue; }
```

HTML

```html
<nav class="navigation">
  <a href="index.html">Startseite</a>
</nav>
<a href="#seitenanfang">Top</a>
```

Das „C“ in CSS – Cascading

Kaskadieren heißt, dass bestimmte styles von einem Obercontainer an seine Unterobjekte weitergereicht werden. Hier eine Liste der Eigenschaften, die auch auf sogenannte Kinder wirken:

color
font-family, font-size, font-weight
line-height
text-align
visibility
cursor

Das „C“ in CSS – Cascading

Wenn wir also im body ein Grundsetup machen mit diesen drei Werten, dann wirken diese Werte ersteinmal auf alle Texte und werden auf diese vererbt. Es sei denn, wir wirken später einen spezifischen Style der diese Werte überschreibt.

```
body {
    font-family: Arial, sans-serif;
    font-size: 16px;
    color: black;
}
```

CSS – HTML Ansicht ohne CSS

Welcome to My Homepage

Use the menu to select different Stylesheets

- Stylesheet 1
- Stylesheet 2
- Stylesheet 3
- Stylesheet 4
- No Stylesheet

Same Page Different Stylesheets

This is a demonstration of how different stylesheets can change the layout of your HTML page. You can change the layout of this page by selecting different stylesheets in the menu, or by selecting one of the following links: Stylesheet1, Stylesheet2, Stylesheet3, Stylesheet4.

No Styles

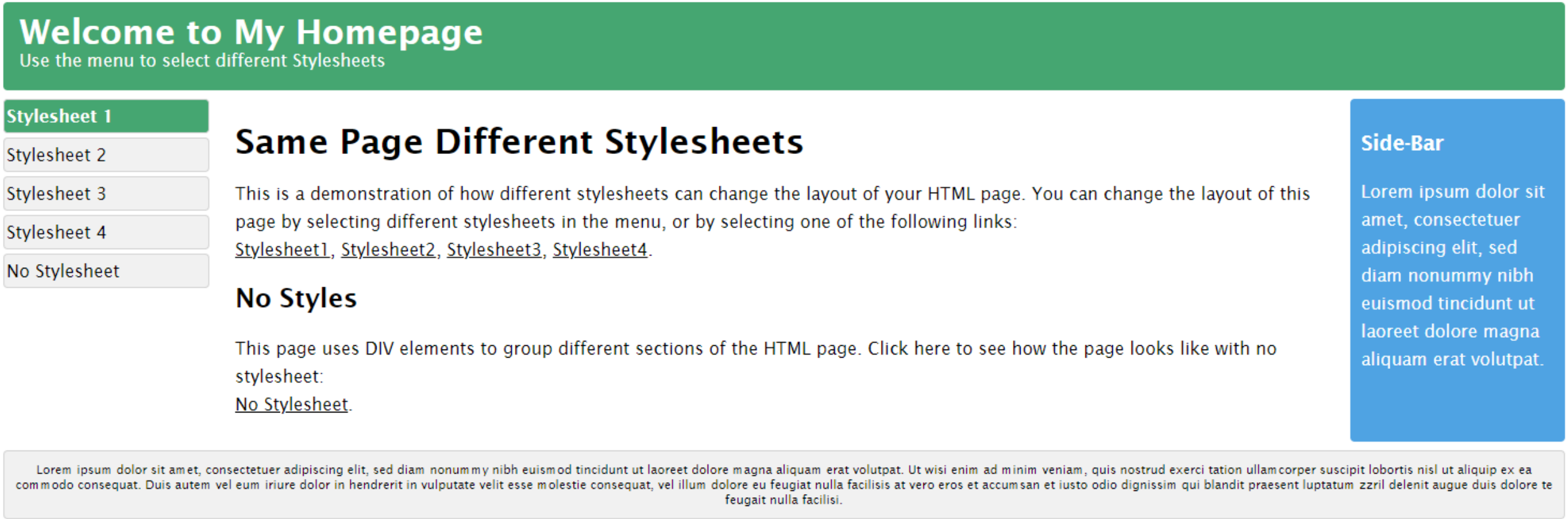
This page uses DIV elements to group different sections of the HTML page. Click here to see how the page looks like with no stylesheet:
No Stylesheet.

Side-Bar

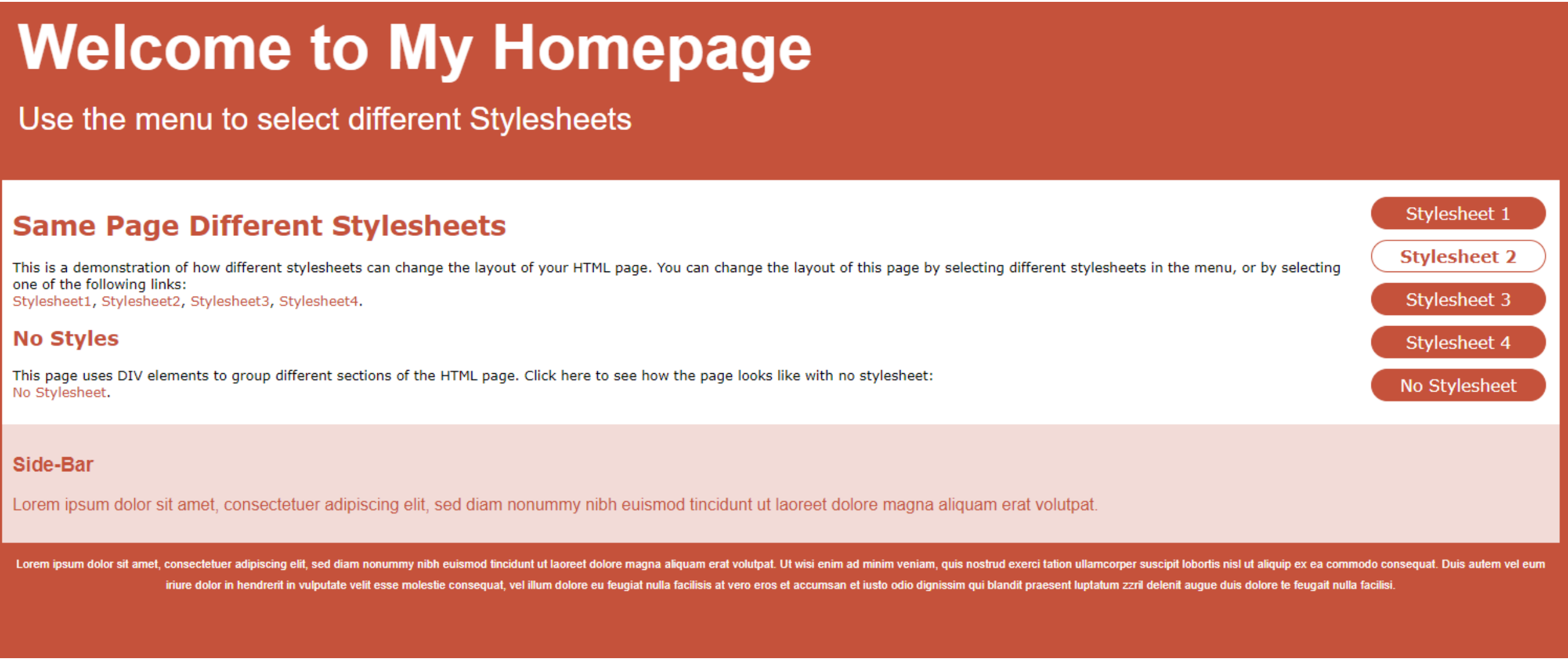
Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue duis dolore te feugait nulla facilisi.

CSS – Stylesheet 1



CSS – Stylesheet 2



CSS – Stylesheet 3

Welcome to My Homepage

Use the menu to select different Stylesheets

Stylesheet 1 | Stylesheet 2 | Stylesheet 3 | Stylesheet 4 | No Stylesheet

Same Page Different Stylesheets

This is a demonstration of how different stylesheets can change the layout of your HTML page. You can change the layout of this page by selecting different stylesheets in the menu, or by selecting one of the following links:
Stylesheet1, Stylesheet2, Stylesheet3, Stylesheet4.

No Styles

This page uses DIV elements to group different sections of the HTML page. Click here to see how the page looks like with no stylesheet:
No Stylesheet.

Side-Bar

Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue duis dolore te feugait nulla facilisi.

Cascading Style Sheets

Implementierung in der .html Datei im <head>.

```
<link rel="stylesheet" type="text/css" href="styles.css">
```

oder auch innerhalb einer HTML Datei in einem <style> ... </style> Tag.

CSS - Grundaufbau & Syntax

Selector Deklaration aus property & value

```
body {
        background-color: blue;
}
```

Wichtig: In CSS werden HTML <tags> nicht mit den <spitzen> Klammern geschrieben.

```
h1, p, div, body {
    color: red;
}
```

CSS - Grundaufbau & Syntax

Selector Deklaration aus property & value

```
body {
        background-color: blue;
}
```

Der Selektor ist das Ziel das wir ansprechen und in den geschwungenen Klammern { … } steht, was wir darauf wirken.

CSS - Beispiele

```
body {      background-color: lightblue;          }

h1 {        color: white; text-align: center;     }

p {         font-family: verdana; font-size: 20px; }
```

Übung

CSS - Welche Selektoren gibt es?

Selector Deklaration aus property & value

```
body {
        background-color: blue;
}
```

CSS - Welche Selektoren gibt es?

CSS kennt folgende einfache Selektoren:

Einfache Selektoren sind:

Universalselektor,

Typselektor,

Klassenselektor,

ID-Selektor,

Attributselektor und Pseudoklassen.

CSS - Welche Selektoren gibt es?

Im Professionellen Umfeld nutzt man Klassen in CSS.

Der Universalselektor wird meistens für einen „Mini-Reset“ benutzt. ID-Selektoren werden sehr selten benutzt.

Deshalb fokussieren wir uns auf die Klassenselektoren.

CSS - Welche Selektoren gibt es?

Das direkte ansprechen von Typselektoren nutzt man nur am Start für ein Grundsetup (body & Headlines).

Um Inhalte zu stylen sollte man niemals Typselektoren ein Verhalten geben sondern eine Klasse nutzen und diese einem HTML tag zuweisen.

CSS - Universalselektor

```
* {margin: 0 }
```

Wirklich jedes Element der Seite erhält einen Außenabstand von 0.

Mehr dazu beim Box Model.

Übung 10.3

CSS - Typ / Element Selektor

```
p  {  font-size: 12px; }
```

Jeder Textabsatz erhält die Schriftgröße 12px.

```
body, p, span, a  {   font-size: 12px; }
```

Es ist auch möglich, mehrere Elemente /Typselektoren zu kombinieren.

CSS - ID Selektor

```
#seitenanfang  {  font-size: 12px; }
```

Der Text des Elements mit der id=“seitenanfang“ erhält die Schriftgröße 12px.

CSS - Klassen Selektor

```
.warnung  { background-color: red; }
```

Alle HTML Elemente mit der class=“warnung“ erhalten einen roten Hintergrund.

CSS - Klassen Selektor

```
.container p                                {       background-color: red;    }
.hauptcontainer .untercontainer .text       {       background-color: red;    }
```

Die übliche Art, wie man CSS im professionellen Umfeld nutzt ist, das man zu stylende Inhalte in abgeschlossenen Boxen oder Container packt.

Hintergrund ist der, dass man so sicherstellt nicht unbedacht andere Formatierungen zu überschreiben.

CSS - Klassen Selektor

```
.warnung  {   background-color: red;  }
```

Alle Elemente mit der class=“warnung“ erhalten einen roten Hintergrund.

Tipp: Im CSS schreibt man die Klassen alle mit Punkt vorweg.
Im HTML innerhalb des Attributes class=“warnung“ wird der Punkt nicht benutzt!

CSS – Selektoren

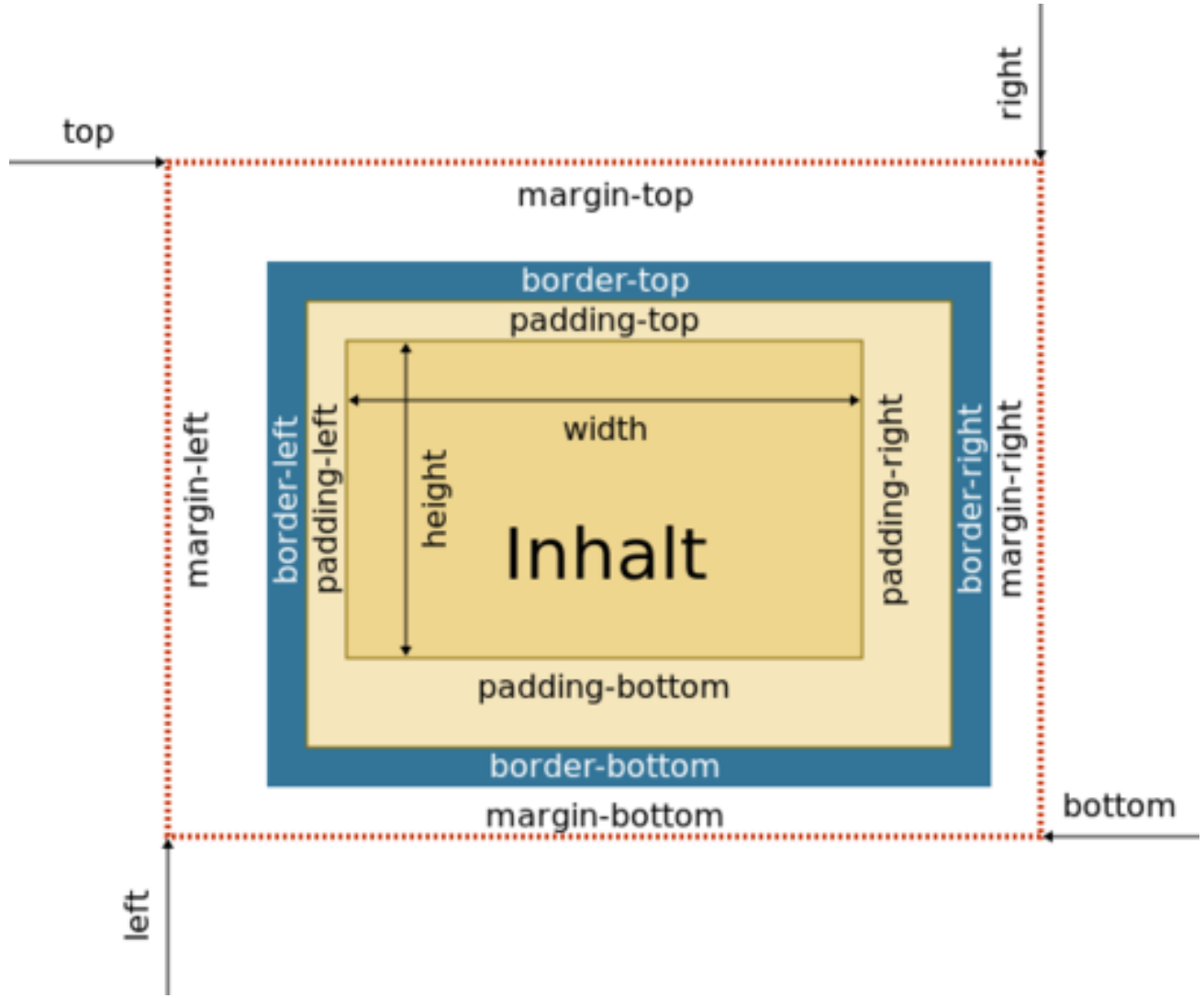
Übung

CSS - Box Model

```
main {
    border: 3px;
    margin: 32px 8px 8px 32px;
    padding: 8px 32px 32px 8px;
    width: 100px;
    height: 50px;
}
```

3 Varianten:

```
margin: 8px 16px 8px 16px;
margin: 8px 16px;
margin: 16px;
```



CSS - Box Model

Die Angabe „box-sizing“ erlaubt uns, das padding (Innenabstand) und den Rahmen in die Gesamtbreite und Höhe mit einfließen zu lassen.

box-sizing: border-box;

width, height
border, padding, margin

CSS - Box Model

Simpler Reset von Abständen

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
```

https://www.w3schools.com/css/css_boxmodel.asp

CSS – Box Model

Übung

CSS – Kombinatoren

Adressierung oder Einschränkung von Styles wann und wo sie wirken.

```
.hauptinhalt p {
    font-family: Arial, Verdana, sans-serif;
}
# Nur wenn p innerhalb eines Containers mit der Klasse .hauptinhalt liegt, wird er angesprochen
```

CSS – Selektoren und Verkettung

Adressierung oder Einschränkung von Styles wann und wo sie wirken.

```
header p {
        font-family: Arial, Verdana, sans-serif;
}

main h2 {
        background-color: red;
}

footer img {
        width: 100%;
}
```

CSS – Selektoren und Verkettung

Adressierung oder Einschränkung von Styles wann und wo sie wirken.

```
header p {
        color: blue;
}
```

Heißt: Nur <p> Tags, die innerhalb eines <header> Tags stehen, werden hier angesprochen und bekommen die Textfarbe blau.

Der zu verändernde Selector ist immer der letzte, also hier das p.

CSS - Kombinatoren (Combinators)

Es gibt 4 verschiedene Kombinatoren.

Nachfahrenselektor (descendant) (space)
Kindselektor (child) (>)
Nachbarselektor (adjacent sibling) (+)
Geschwisterselektor (general sibling) (~)

CSS - Nachfahrenselektor (space)

```
.container p {
    background-color: yellow;
}
```

Hiermit werden alle <p> tags in einer „Box“ mit der Klasse class="container„ angesprochen.

```
<div class="container">
  <p>Paragraph 1 in the div.</p>
  <p>Paragraph 2 in the div.</p>
  <section><p>Paragraph 3 in the div.</p></section>
</div>

<p>Paragraph 4. Not in a div.</p>
```

CSS - Kindselektor (>)

```
.container > p {
    background-color: yellow;
}
```

Hiermit werden nur alle „direkten Nachkommen“ eines <p> tags im tag mit der Klasse class="container" angesprochen.

```
<div class="container">
  <p>Paragraph 1 in the div.</p>
  <p>Paragraph 2 in the div.</p>
    <span><p>nope</p></span>
</div>
```

CSS – Selektoren & Kombinatoren

https://www.w3schools.com/css/css_combinators.asp

CSS – Kombinatoren

Übung

CSS - Attributselektoren

Der Attributpräsenz-Selektor spricht alle Elemente an, die ein bestimmtes Attribut besitzen.

Varianten sind Attributspräsenz, Attributwert und Teilübereinstimmung.

CSS - Attributselektoren Präsenz

Der Attributpräsenz-Selektor spricht alle Elemente an, die ein bestimmtes Attribut besitzen.

```
[href] {      border: 1px solid black;      }
```

Alle Elemente, die ein href-Attribut besitzen bekommen einen 1px Rahmen.

CSS - Attributselektoren Wert

Der Attributswert-Selektor spricht alle Elemente an, die ein bestimmtes Attribut mit einem bestimmten Wert besitzen.

```
[char="="] {     font-family: monospace; }
```

Alle Elemente, deren Inhalt am Gleichheitszeichen ausgerichtet wird, werden in monospace dargestellt.

CSS - Attributselektoren Teilübereinstimmung

Der Teilübereinstimmung-Selektor spricht alle Elemente mit einem bestimmten Attribut an, dessen Attributwert ein bestimmtes Wort enthält.

[attributname^="wort"] (Anfang)
[attributname$="wort"] (Ende)
[attributname*="wort"] (enthält)

```
a[href^="https://"] { font-weight: bold }
a[href$=".pdf"]     { color: green }
a[href*="wiki"]     { font-style: italic }
```

CSS - Attributselektoren Teilübereinstimmung

```
a[href^="https://"] { font-weight: bold }
```

```
<ul>
  <li><a href="https://example.com/">Beispiellink</a></li>
  <li><a href="https://example.com/example.pdf">Ein PDF-Dokument</a></li>
  <li><a href="https://wiki.selfhtml.org/">Das selfhtml wiki</a></li>
  <li><a href="http://example.org/WIKI/example.pdf/">kein Selektor darf diesen Link
stylen.</a></li>
</ul>
```

Cascading Style Sheets

Das „Cascading“ heißt, das bestimmte Styles von einem Übergeordneten <html tag> nach „unten“ weiter gereicht werden.

Ein gutes Beispiel dafür: Schriftzuweisungen.

Cascading Style Sheets

```
html {
    font-family: Arial;
}
```

Für das gesamte Dokument auf das dieser CSS-Style wirkt wird vorab die Schriftart auf Arial gesetzt.

Es sei denn, ich überschreibe diesen Stile später (z.B. in der nächsten Zeile Code) für einen spezifischen tag mit etwas anderem.

Cascading Style Sheets

```
html {
    font-family: Arial;
}

h1 {
    font-family: Verdana;
}
```

JEDER Text (im Dokument) wird jetzt mit dem Zeichensatz Arial dargestellt. Bis auf die <h1>. Die bekommt den Font: Verdana zugewiesen.

Cascading Style Sheets

```
html {
    font-family: Arial;
}

h1 {
    font-family: Verdana;
}
```

Das ist auch das Grundkonzerpt von CSS. Man fängt „im Groben“ an und wird dann immer spezifischer (also genauer).

Cascading Style Sheets

Das ist auch das Grundkonzerpt von CSS. Man fängt „im Groben“ an und wird dann immer spezifischer (also genauer).

Das heißt:

Man definiert im body ein „Grundsetup“ oder die „Voreinstellung“ und danach definiert man Außnahmen.

Cascading Style Sheets

```
html {
    font-family: Arial;
}

h1 {
    font-family: Verdana;
}

.besonderer-text {
    font-family: Georgia;
}
```

Jeder Text formatiert in Arial.

Die <h1> in Verdana.

Ein <p class=„besonderer-text“> nutzt dann den Zeichensatz: Georgia.

Cascading Style Sheets

Font Zuweisungen sind nicht die einzigen Eigenschaften die vererbt werden. Hier eine Liste der Eigenschaften, die auch auf sogenannte Kinder wirken:

color
font-family
font-size
font-weight
line-height
text-align
visibility
cursor

Cascading Style Sheets

Beispiel:

```
body {
    color: red;
}
```

Jeder Text im <body> bekommt die Textfarbe rot.

```
article {
    visibility: hidden; /* (versteckt) */
}
```

Alles was in dem article ist wird unsichtbar, nicht nur der Container <article> selbst.

Beispiel & Übung

Sonnensystem

Block & Inline Elemente

Wer hat Platz und wer verschafft sich welchen?

Dies ist ein übergreifendes Thema was HTML & CSS betrifft.

Block & Inline Elemente

Wer hat Platz und wer verschafft sich welchen?

Block-Elemente nehmen sich immer den maximalen Platz in der Breite den sie kriegen können. Sie springen „in die nächste Zeile“ wenn vor ihnen schon ein Element ist.

Inline-Elemente oder Container kann man sich wie ein Gummiband vorstellen. Die Größe des Containers wird durch den Inhalt „aufgedrückt“, mit anderen Worten der Container versucht so klein wie möglich zu sein.

Block Elemente

Elemente, die immer in einer neuen Zeile beginnen, die gesamte Breite einnehmen und „sich Platz schaffen“, also nachfolgende Elemente in die nächste Zeile drängen.

Block Elemente

<address><article><aside><blockquote><canvas><dd>
<dl><dt><fieldset><figcaption><figure>
<footer><form><h1> bis <h6>
<header><hr>
<main><nav>
<noscript>
<p><pre><section>
<table><tfoot>
<video>

Inline Elemente

Inline Elemente nutzen nur exakt den Bereich, den ihr Inhalt benötigt, fahren also nicht „die Ellenbogen aus“ und beanspruchen die gesamte Breite.

Inline Elemente

<a><abbr><acronym><b><bdo><big>
<button>
<cite><code><dfn><i><img><input><kbd><label><map><object>
<output><q><samp><script><select>
<small><sub><sup><textarea><time><tt><var>

Sonderfall: Inline-Block

Im Vergleich zu display: inline kann ich mit display: inline-block dem Element eine Höhe und Breite geben, ebenso wie margins und padding.

Aber im Gegensatz zu display: block wird kein Zeilenumbruch durgeführt.

display: none

Eine zusätzliche Option von display ist „none“.

```
.versteckmich {
    display: none;
}
```

Der Container mit der Klasse .versteckmich wird nicht angezeigt.

CSS – Block & Inline

Übung

CSS - Ausrichtung mit position

Elemente können durch Verwendung der Eigenschaft position aus dem normalen Elementfluss entfernt werden und an jede beliebige Stelle des Viewports positioniert werden. Auf andere Elemente haben positionierte Elemente keinen Einfluss.

CSS - Ausrichtung mit position

Folgende Werte sind möglich:

static (Standardwert)
relative
absolute
fixed
sticky

CSS - Ausrichtung mit position: static

position: static ist der Standardwert, den jedes HTML-Element automatisch besitzt. Das HTML-Element befindet sich folglich an seiner Standardposition innerhalb des sog. Dokumentflusses. Die später beschriebenen Eigenschaften left, right, top & bottom, sowie der z-index haben bei einem Element mit position: static keinerlei Wirkung. Man spricht deshalb auch von einem nicht positionierten Element.

Der Wert static wird normalerweise dann verwendet, wenn man ein Element zuvor mit einer der folgenden position-Werte positioniert hat und den Standardzustand wiederherstellen möchte.

CSS - Ausrichtung mit position: relative

Wenn ihr einem Element position:relative zuweist, positioniert es sich relativ zu seiner Standard-Position, also jener Stelle auf der Website, die es ohne Positionierungsangaben einnehmen würde. Wir können das Element mithilfe der CSS-Eigenschaften left, right, top & bottom von dort aus verschieben.

Andere Elemente berücksichtigen bei ihrer Positionierung weiterhin die ursprüngliche Position des verschobenen Elements, nicht die sichtbare.

CSS - Ausrichtung mit position: absolute

Wenn ihr ein Element mit position:absolute; positioniert, orientiert es sich am nächst höheren Eltern-Element, das nicht den Standardwert position: static trägt. Gibt es ein solches Element nicht, orientiert es sich am HTML-Element body.

Das absolut positionierte Element wird darüber hinaus dem Dokumentfluss entnommen. Andere Elementen rutschen daher in den freigewordenen Platz.

CSS - Ausrichtung mit position: fixed

Wenn ihr ein Element mit position: fixed; positioniert, orientiert es sich am Browserfenster. Wie bei position:absolute; wird es dem Dokumentfluss entnommen, was dazu führt, dass nachfolgende Elemente in den freigewordenen Platz rutschen. Zusätzlich sorgt position: fixed dafür, dass das Element beim Scrollen an Ort und Stelle bleibt. So kann man z. B. eine Navigationsleiste für den Nutzer permanent sichtbar machen.

CSS - Ausrichtung mit position: sticky

Die Positionierungsart position: sticky ist ein Hybrid der zwei Eigenschaften position: relative und position: fixed. Das Element scrollt solange mit, bis es mit einer seiner Kanten eine der über top, right, bottom oder left angegebenen Koordinaten erreicht. Üblicherweise wird die top-Eigenschaft verwendet, da wir im Regelfall eine Website von oben nach unten scrollen.

Sobald der Wert erreicht ist, verhält sich das Element als wäre es mit position:fixed; positioniert worden. Es bleibt also beim Scrollen stehen.

CSS - Ausrichtung mit position

Eigenschaften in Verbindung mit position:

left, right, top & bottom, sowie der z-index

https://www.w3schools.com/css/css_positioning.asp

CSS – Position

Übung

CSS - Spezifität

Wer oder was hat in der CSS Vererbungshirarchie Vorrang?

https://wiki.selfhtml.org/wiki/CSS/Tutorials/Einstieg/Kaskade#Spezifit.C3.A4t

CSS - Extern

CSS Reset oder normalize.css

https://meyerweb.com/eric/tools/css/reset/

https://necolas.github.io/normalize.css/

CSS - Spezialthemen

Background Property